

Stream#

<https://pytorch.org/docs/stable/generated/torch.Stream.html#torch.Stream>

Description:

An in-order queue of executing the respective tasks asynchronously in first in first out (FIFO) order. It can control or synchronize the execution of other Stream or block the current host thread to ensure the correct task sequencing. It supports with statement as a context manager to ensure the operators within the with block are running on the corresponding stream. See in-depth description of the CUDA behavior at CUDA semantics for details on the exact semantic that applies to all devices. device (torch.device, optional) – the desired device for the Stream. If not given, the current accelerator type will be used. priority (int, optional) – priority of the stream, should be 0 or negative, where negative numbers indicate higher priority. By default, streams have priority 0. An torch.Stream object.

Function Signature

```
class torch.Stream(device, *, priority)#
```

Parameters

Returns

Return type

```
query() → bool#
```

Returns:

Event

Code Examples

```
>>> with torch.Stream(device='cuda') as s_cuda:
>>>     a = torch.randn(10, 5, device='cuda')
>>>     b = torch.randn(5, 10, device='cuda')
>>>     c = torch.mm(a, b)
```

```
>>> s_cuda = torch.Stream(device='cuda')
>>> s_cuda.query()
True
```

```
>>> s_cuda = torch.Stream(device='cuda')
>>> e_cuda = s_cuda.record_event()
```

```
>>> s_cuda = torch.Stream(device='cuda')
>>> s_cuda.synchronize()
```

```
>>> s1_cuda = torch.Stream(device='cuda')
>>> s2_cuda = torch.Stream(device='cuda')
>>> e_cuda = s1_cuda.record_event()
>>> s2_cuda.wait_event(e_cuda)
```

```
>>> s1_cuda = torch.Stream(device='cuda')
>>> s2_cuda = torch.Stream(device='cuda')
>>> s2_cuda.wait_stream(s1_cuda)
```

Detailed Documentation

Documentation

An in-order queue of executing the respective tasks asynchronously in first in first out (FIFO) order. It can control or synchronize the execution of other Stream or block the current host thread to ensure the correct task sequencing. It supports with statement as a context manager to ensure the operators within the with block are running on the corresponding stream.

See in-depth description of the CUDA behavior at [CUDA semantics](#) for details on the exact semantic that applies to all devices.

`device` (`torch.device`, optional) – the desired device for the Stream. If not given, the current accelerator type will be used.

`priority` (int, optional) – priority of the stream, should be 0 or negative, where negative numbers indicate higher priority. By default, streams have priority 0.

An `torch.Stream` object.

Check if all the work submitted has been completed.

A boolean indicating if all kernels in this stream are completed.

Record an event. En-queuing it into the Stream to allow further synchronization from the current point in the FIFO queue.

`event` (`torch.Event`, optional) – event to record. If not given, a new one will be allocated.

Wait for all the kernels in this stream to complete.

Make all future work submitted to the stream wait for an event.

`event` (`torch.Event`) – an event to wait for.

Synchronize with another stream. All future work submitted to this stream will wait until all kernels already submitted to the given stream are completed.

`stream` (`torch.Stream`) – a stream to synchronize.

